

IMPERIAL

Department of Civil and
Environmental Engineering
FACULTY OF ENGINEERING

Introduction to Structural Analysis

Dr Andrew Phillips

Reader in Structural Biomechanics

While intended to give an overview of the content of the module these notes are not a complete record and should not be treated as a substitute for participating in lectures and tutorials, undertaking private study, and completing the coursework.

© Copyright 2026 Andrew Phillips

Email: andrew.phillips@imperial.ac.uk



This work is distributed under a Creative Commons
Attribution Non-Commercial Share-Alike license: CC BY-NC-SA 4.0

GitHub repository | DOI: [10.5281/zenodo.19269579](https://doi.org/10.5281/zenodo.19269579)

1 Introduction

This part of the module investigates how the stiffness method can be applied to allow us to carry out structural analysis. The stiffness method is a subtype of the finite element method.

The stiffness method is implemented in Python, while Grasshopper (the parametric design plugin in Rhino) is used to develop structural models and to visualise the results from the structural analysis carried out in Python. Rather than using a separate Python environment (such as Spyder in the Anaconda distribution) we will use a Python component in Grasshopper, which with care allows data to be passed between Python and Grasshopper, so that pre-processing, structural analysis, and post-processing can be carried within a Grasshopper definition, allowing the effect of altering a model parameter to be easily assessed.

The stiffness method using Euler-Bernoulli beam elements in 2D, and the process of structural analysis is described in these notes, while video tutorials guide you through its application in Python and Grasshopper.

The stiffness and finite element methods are based on the relationship:

$$\underline{\underline{f}} = \underline{\underline{K}}\underline{\underline{u}} \quad (1.1)$$

where $\underline{\underline{f}}$ is a column matrix ($n \times 1$) of external forces, $\underline{\underline{u}}$ is a column matrix ($n \times 1$) of displacements, and $\underline{\underline{K}}$ is a square matrix ($n \times n$) referred to as the structural or system stiffness matrix, where n is the number of nodal degrees of freedom in a structural model. In Equation 1.1 the underscore tilde ($\underline{\underline{\quad}}$) is used to indicate that the parameters are matrices. For convenience we will often omit this in these notes.

In 2D each node has three degrees of freedom, two translational degrees of freedom in the x and y axes, and a rotational degree of freedom about the z axis.

You can download a 90-day evaluation version of Rhino, which includes Grasshopper if you wish to use it while you are not on the Imperial network.

 [Download Rhino](#)

While the module material will cover what is required, you may also want to gain further experience in Rhino and Grasshopper using the playlist available on the Civil Engineering Design at Imperial YouTube channel.

 [Introduction to Rhino and Grasshopper playlist](#)

The video tutorials accompanying the module are also available as a playlist on the YouTube channel.

 [Introduction to Structural Analysis using Grasshopper and Python playlist](#)

2 Structural Analysis | Part 1

While we will be using the stiffness method with Euler-Bernoulli beam elements in 2D, the stages to develop a structural model for analysis are the same for any finite element analysis, and are similar for other types of structural analysis. The stages to develop a structural model may be summarised as:

- Define the geometry.
- Define the material properties.
- Define the cross-section properties.
- Define the architecture of the mesh (nodes and elements):
 - Nodes are defined as points with a unique node number and the coordinates of the points.
 - Elements are defined as the connections between nodes with a unique element number and an ordered list of node numbers.
- Define the boundary conditions.
- Define the applied forces (or displacements).
- Construct the system stiffness matrix (K) based on the previous stages.
- Solve to find the displacement matrix (u) using $u = K^{-1}f$ or other solution techniques.
- Solve to find the force matrix (f) including reactions using $f = Ku$.

Finite element analysis does not have in-built units of length or force, so we need to take care to ensure that we use consistent units throughout, so that we know what units our outputs will be in. On this module we will use millimetres (mm) as the units of length (mm^2 as the units of area, mm^3 as the units of volume, and mm^4 as the units of second moment of area) and Newtons (N) as the units of force. This results in units of mm for translational displacements, radians for rotational displacement, N for forces, Nmm for moment, and MPa (N/mm^2) for stresses.

2.1 Geometry

Finite element analysis can be carried out using continuum or structural elements, in 2D or 3D. Continuum elements such as tetrahedral or hexahedral elements are often used in 3D to model complex geometries, while triangular and quadrilateral elements are often used in 2D. On this module we will focus on the use of beam elements in 2D, hence when we define our mesh, we are simultaneously defining our geometry. However, it is useful to be aware of geometry definition as an important stage in finite element analysis.

2.2 Material properties

On this module we will limit ourselves to using isotropic linear elastic material properties. The use of Euler-Bernoulli beam elements in 2D means that the only material property we need to define is Young's modulus (E) which we will define in MPa (N/mm^2).

If we wanted to extend our analyses to 3D we would also need to define the shear modulus (G) based on the Young's modulus and Poisson's ratio (ν) for an isotropic material:

$$G = \frac{E}{2(1 + \nu)} \quad (2.1)$$

Although we will not consider it, a Timoshenko beam formulation, which includes additional rotational deformation about the cross-section y axis due to the shear force V_z , would also require the shear modulus and Poisson's ratio to be defined.

2.3 Cross-section properties

On this module the use of Euler-Bernoulli beam elements in 2D means that the only cross-section properties we need to define are the cross-section area (A) which we will define in mm^2 , and the second moment of area about the y axis (I_{yy}) which we will define in mm^4 . We will use the cross-section axes that are adopted in the Eurocodes and on the relevant part of the Structural Mechanics thread as shown in Figure 2.1. It is assumed that the y axis is a principal bending axis, generally taken as the major bending axis, so $I_{yy} = I_{\max} = I_{uu}$.

Note that if a rectangular cross-section is used in 2D and the breadth (b) is set to a unit value of 1mm, then A and I_{yy} depend only on the depth (d), and d may be relabelled as

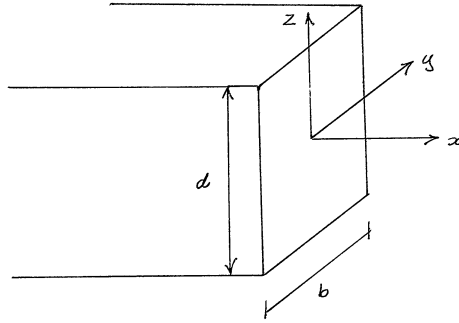


Figure 2.1: Cross-section axis system, also referred to as the beam axis system.

thickness (t).

If we wanted to extend our analyses to 3D we would also need to define the second moment of area about the z axis (I_{zz}), assuming that the z axis is a principal bending axis, generally taken as the minor bending axis, so $I_{zz} = I_{\min} = I_{yy}$, and the torsion constant about the x axis (J).

2.4 Mesh

The architecture of the mesh defines how the nodes and elements are arranged to represent the structure, and may also be referred to as the structural layout or topology of the mesh. When defining a mesh to represent a beam (frame) structure there must be a node at each joint and support in the structure, with at least one element representing each beam (structural member) connecting between joints or supports in the structure.

Depending on the beam element formulation (which we will only briefly discuss here) each beam element is either defined using two or three nodes. For the Euler-Bernoulli beam formulation used to develop the stiffness matrix that you are familiar with from the Structural Mechanics thread we require two nodes, at the start and end of the element. For the Timoshenko beam formulation used in many finite element packages (including Abaqus) three nodes are required, with nodes at the start, middle, and end of the element. The reason for this is that the stiffness matrix defined using the finite element method with an Euler-Bernoulli formulation is generally defined using a Hermite cubic shape function, which can be defined based on two nodes, while the stiffness matrix defined using the finite element method with a Timoshenko formulation is generally defined based on a quadratic shape function, which requires three nodes to define it.

The stiffness matrix defined using an Euler-Bernoulli formulation with a Hermite cubic shape function using the finite element method is identical to that defined using the stiffness method. The finite element formulation is presented in supplementary material in the appendix, and is not discussed further here. It is possible to adapt the stiffness matrix developed using the stiffness method to include shear deformation.

It is common for mesh refinement to be required in order to get a more accurate prediction of the deformation of a structure subject to the applied forces acting on it. This involves subdividing beams (or other structural members) so that several elements are used to represent them. This type of mesh refinement can be referred to as h-refinement, while the use of higher order shape functions in element formulations can be referred to as p-refinement. Mesh or h-refinement may be thought of in terms of mesh resolution, with a coarse mesh having a relatively low mesh resolution (a small number of nodes and elements) and a refined mesh having a relatively high mesh resolution (a large number of nodes and elements). A further type of refinement, referred to as r-refinement involves selectively increasing the number of elements in parts of the mesh where increased accuracy is required.

Grasshopper is well suited for developing structural meshes as it is effective and efficient in generating topologies that define unique sets of nodes and elements. For this reason it is often used as a pre-processor to generate structural models that can be analysed using finite element packages (including Abaqus).

On this module we will use Grasshopper to define a mesh using unique sets of nodes and elements, and to define element length (L), and element angle (α), for each element, which are required to define the system stiffness matrix (K) in 2D.

If we wanted to extend our analyses to 3D we would also need to define the cross-section axis system for each element, so that the orientation of the cross-section (or beam) y and z axes in 3D space where known. In 2D we will assume that the cross-section (or beam) x axis is defined by a vector running between the start and the end node (also the case for the 3D), the y axis is defined by a vector going into the page perpendicular to the plane defined by the page, and the z axis is defined by the cross product of the vectors defining the x and y axes in a right-handed Cartesian axis system.

2.5 Boundary conditions

In this module we will limit ourselves to defining boundary conditions where the displacement in the constrained nodal degree of freedom (the acronyms DoF or dof are often used) is set to zero, meaning that we assume any support is rigid (infinitely or sufficiently stiff). It is possible to apply a similar approach to set a stiffness value in the restrained degree of freedom. Boundary conditions are set in the global axis system, with is discussed further when constructing the system stiffness matrix (K).

In 2D each node has three degrees of freedom, two translational degrees of freedom in the global x and y axes, and a rotational degree of freedom about the global z axis; assuming that in the global axis system the xy plane is the plane of interest, with the z axis defined by a vector going out of the page perpendicular to the $x y$ plane defined by the page in a right-handed Cartesian axis system.

If we wanted to extend our analyses to 3D each node would have six degrees of freedom, three translational degrees of freedom in the global x , y , and z axes, and three rotational degrees of freedom about the global x , y , and z axes.

2.6 Applied forces

Applied forces are external forces applied as point forces (including moments) at the nodes, and like boundary conditions are set in the global axis system. These forces are defined in the force matrix (f) which is an $n \times 1$ column matrix, where n is the total number of nodal degrees of freedom in the structural model. In 2D n is equal to three times the number of nodes, while in 3D n is equal to six times the number of nodes.

Distributed loads can be included as applied forces but are included as equivalent nodal loads rather than as loads applied directly to an element. Equivalent nodal loads calculated in a local (beam) axis system need to be resolved into forces in the global axis system for inclusion in the f matrix. For a uniformly loaded beam running parallel to the global x axis, with a uniformly distributed load acting downwards (in the negative y direction) the equivalent nodal loads are shown in Figure 2.2. It is observed that the equivalent nodal loads are the negative of the fixed end reactions that you will have encountered when using the stiffness method for hand-written calculations.

Reaction forces at constrained degrees of freedom are applied forces, but do not need to

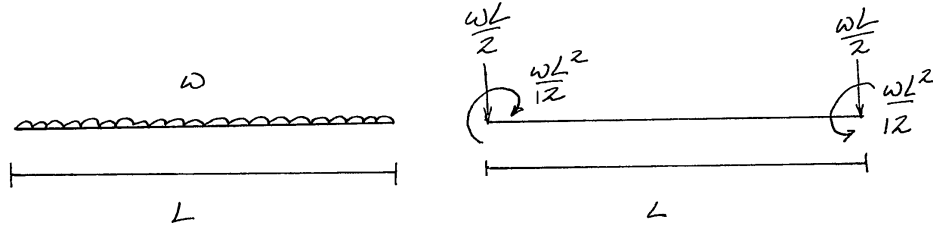


Figure 2.2: Equivalent nodal loads for a uniformly distributed load acting along the length of a beam.

be included in the constructed f matrix, as they will be found as part of the analysis.

Applied displacements can also be included through alterations to the stiffness matrix (K) and force matrix (f) when these are constructed, ensuring that the relevant term in the displacement matrix (u) has the desired value.

2.7 Constructing the system stiffness matrix

To construct the system stiffness matrix (K) we use the information provided by the elements and nodes in the mesh. For each element we can first calculate an element stiffness matrix (k) which for a beam element in 2D will be a 6×6 square matrix with the terms in the matrix providing the relationship between the displacements at the start and end nodes, and the applied forces at the start and end nodes, for each element. Hence we will have the same number of k matrices as we have elements. In Python we can store these in an $n_e \times 6 \times 6$ NumPy array, where n_e is the number of elements.

For convenience we will adopt the local axis system shown in Figure 2.3. If the element defines the direction of the local x axis, then the local y axis is in the same direction as the cross-section z axis, and the local z axis is in the opposite direction to the cross-section y axis. This will be important when we post-process the results from the analysis to allow us to plot axial force (F_x), shear force (V_z), and bending moment (M_y) diagrams, and the distribution of normal stress (σ_{xx}) in the cross-section axis system.

The 6×6 element stiffness matrix (k) for an Euler-Bernoulli beam in 2D is:

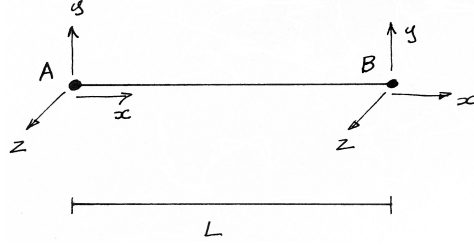


Figure 2.3: Local axis system at the start (A) and end (B) nodes for each element.

$$\begin{bmatrix} R_{xA} \\ R_{yA} \\ M_A \\ R_{xB} \\ R_{yB} \\ M_B \end{bmatrix} = \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ \hline -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \theta_A \\ \delta_{xB} \\ \delta_{yB} \\ \theta_B \end{bmatrix} \quad (2.2)$$

where the element force (f) and displacement (u) 6×1 column matrices are also shown. The element stiffness matrix (k) gives the relationships between displacements and forces at the start and end nodes as shown in Figure 2.4. I is I_{yy} in the cross-section axis system and θ is θ_z in the local and global axis systems.

The 6×6 k matrix shown in Equation 2.2 is formed of four 3×3 quadrants, which are of importance when constructing the system stiffness matrix (K):

- The top-left quadrant relates forces at node A to displacements at node A.
- The bottom-right quadrant relates forces at node B to displacements at node B.
- The top-right quadrant relates forces at node A to displacements at node B.
- The bottom-left quadrant relates forces at node B to displacements at node A.

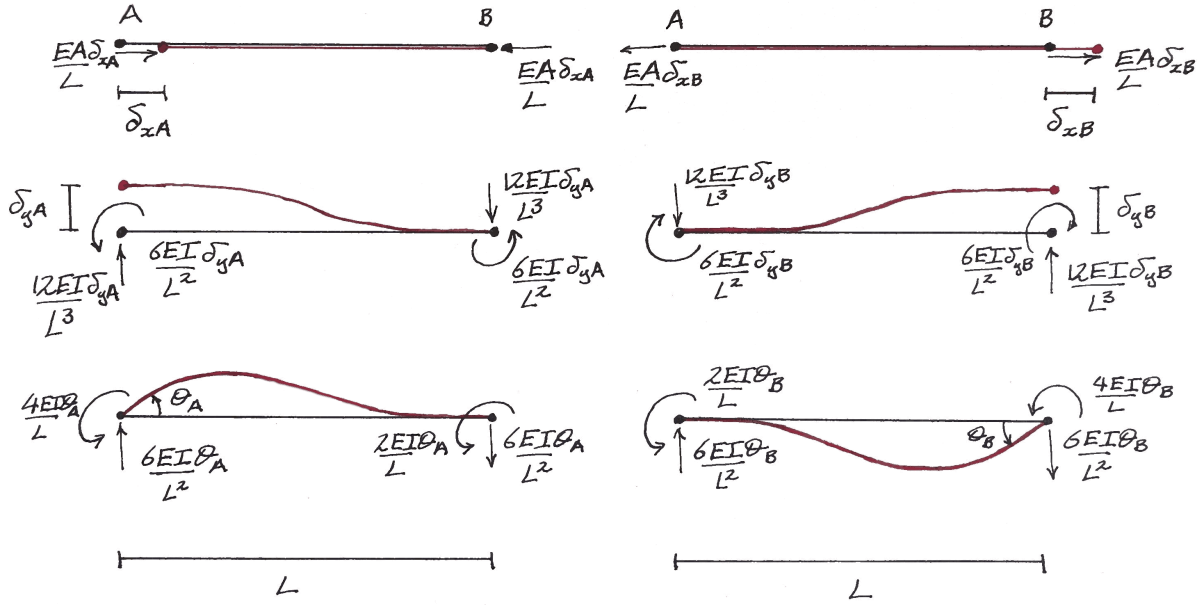


Figure 2.4: Relationships defined by terms in the element stiffness matrix (k) between displacements (u) and forces (f) at the start (A) and end (B) nodes of an Euler-Bernoulli beam element in 2D. The displaced shapes of the beams are sketched.

Before we construct the system stiffness matrix (K) from the element stiffness matrices (k) we need to consider how the element stiffness matrix should be transformed if the element is at an angle (α) to the global axis system, as shown in Figure 2.5.

Each element stiffness matrix (k) can be transformed into an element stiffness matrix in the global axis system (k') in the same way as a second order tensor.

$$k' = N^T k N \quad (2.3)$$

where N is the direction cosine matrix, and N^T is its transpose. The direction cosine matrix is formed of the basic unit:

$$N = \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix} \quad (2.4)$$

To transform k to k' the basic unit is used to define a 6×6 matrix:

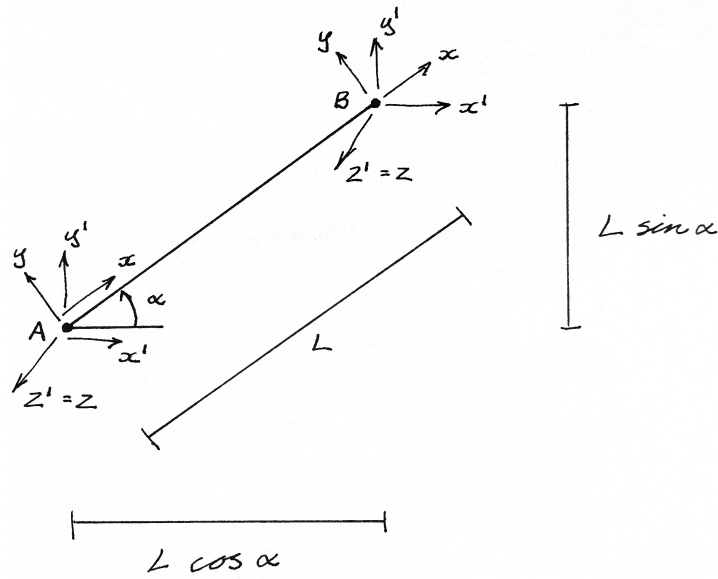


Figure 2.5: Angle (α) that the local axis system (xyz) is rotated by in the global axis system ($x'y'z'$).

$$N = \left[\begin{array}{ccc|ccc} \cos \alpha & \sin \alpha & 0 & 0 & 0 & 0 \\ -\sin \alpha & \cos \alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & \cos \alpha & \sin \alpha & 0 \\ 0 & 0 & 0 & -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right] \quad (2.5)$$

The four quadrants of k' relate the same displacements and forces for node A and node B as they did for k .

The system stiffness matrix (K) is an $n \times n$, or $3n_n \times 3n_n$ matrix, where n is the number of nodal degrees of freedom, and n_n is the number of nodes, with each node in a 2D structural model having three degrees of freedom. The four quadrants of each element stiffness matrix in the global axis system (k') are added to the K matrix based on the node numbers given in the element definition.

Python and Grasshopper use **zero-based indexing** (which differs to MATLAB for example). Python uses **right-hand-side half-open interval indexing** ($i \in [0 : N)$, $0 \leq i < N$). This means that for an element with node A numbered 0 and node B numbered 2, in Python the four 3×3 quadrants of k' would be added to the indexed locations in K :

- The top-left quadrant of k' added to rows and columns $[0:3,0:3]$ in K .
- The bottom-right quadrant of k' added to rows and columns $[6:9,6:9]$ in K .
- The top-right quadrant of k' added to rows and columns $[0:3,6:9]$ in K .
- The bottom-left quadrant of k' added to rows and columns $[6:9,0:3]$ in K .

The process of adding the k' matrix quadrants to the K matrix is carried out for each element until the completed system stiffness matrix (K) is constructed. However, to use the K matrix in analysis we need to adjust it based on any boundary conditions and applied displacements that we wish to include.

2.7.1 Adjusting for boundary conditions

To constrain a nodal degree of freedom we will use a penalty approach. Where CDoF is the index of the constrained degree of freedom this involves:

- Setting all the row ($[CDoF, :]$) terms to zero.
- Setting all the column ($[:, CDoF]$) terms to zero.
- Assigning the diagonal term ($[CDoF, CDoF]$) an extremely high stiffness, for example $K_{\max} \times 10^6$ where $K_{\max}$ is the absolute maximum term in the K matrix prior to adjustment.

This results in the force required to produce a displacement close to zero (an infinite stiffness would result in a displacement of zero) being found when the system is solved.

2.7.2 Adjusting for applied displacements

To introduce an applied displacement we can also use a penalty approach. Where DDoF is the index of the displaced degree of freedom this involves:

- Assigning the diagonal term ($[DDoF, DDoF]$) an extremely high stiffness, for example $K_{\max} \times 10^6$ where $K_{\max}$ is the absolute maximum term in the K matrix prior to adjustment.

- Altering the DDoF term in the force matrix (f) for example to $\delta K_{\max} \times 10^6$ where δ is the applied displacement.

This results in a displacement close to the desired value being found when the system is solved.

2.8 Solve to find the displacement and force matrices

Once the system stiffness (K) and force (f) matrices have been constructed we can solve to find the displacement (u) matrix. While this can be done by finding the inverse of K :

$$u = K^{-1}f \quad (2.6)$$

this is not the most efficient way to solve for u . For reasons we do not need to consider, in Python it is more efficient to use the `solve` function available from the linear algebra functions in the SciPy package.

When the displacement matrix (u) has been found the force matrix (f) including reaction forces can be found using the unadjusted K matrix:

$$f = Ku \quad (2.7)$$

2.9 Deriving beam end forces

While the displacements and forces at the nodes are produced when solving for the u and f matrices, we are also interested in finding the beam end forces to allow us to plot the axial force (F_x), shear force (V_z), and bending moment (M_y) diagrams in the cross-section axis system for each beam element. To do this we can find the beam end forces for each element in its local axis system based on its orientation in the global axis system, which we will implement in Python, before converting from the local axis system to the cross-section axis system, which we will implement in Grasshopper.

To find the beam end forces we can find the node displacements in the global axis system for node A and node B for each element, transform these into the local axis system, and

multiply the resulting matrix by the element stiffness matrix (k). This sequence can be summarised as:

$$f_{en} = kNu_{en} \quad (2.8)$$

where f_{en} is a 6×1 element node force matrix (beam end forces) in the local axis system, and u_{en} is a 6×1 element node displacement matrix in the global axis system. Once we have the beam end forces we can convert these to internal forces in the beam based on the positive conventions shown in Figure 2.6, where we are looking at a cross-section to the right-hand-side of a cut at node A, and at a cross-section to the left-hand-side of the cut at node B.

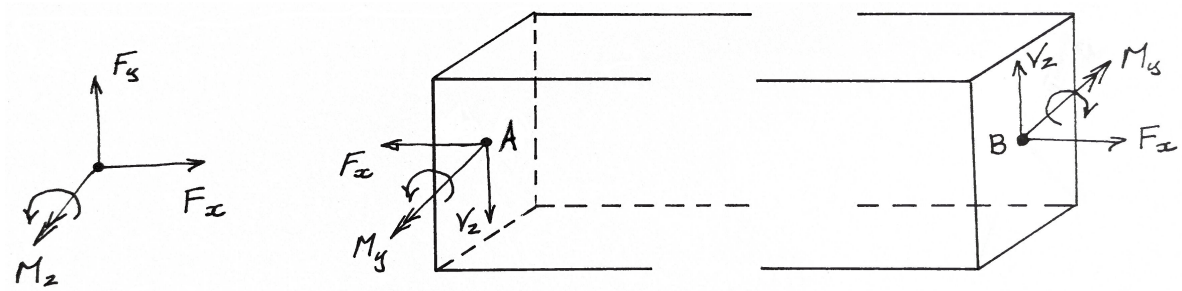


Figure 2.6: Positive force diagram conventions in the local axis system (shown here in the same orientation as the global axis system), and at node A and node B in the cross-section axis system, for a beam element.

This gives us the conversions between beam end forces in the local axis system and internal forces in the cross-section axis system in Table 1.

Internal force	Node A	Node B
Axial force (F_x)	$-F_x$	$+F_x$
Shear force (V_z)	$-F_y$	$+F_y$
Bending moment (M_y)	$+M_z$	$-M_z$

Table 1: Conversion between beam end forces in the local axis system, and internal forces at node A and node B in the cross-section axis system, for a beam element.

3 Grasshopper and Python

On the module we will be using Grasshopper for pre- and post-processing, and the Python component in Grasshopper to carry out structural analysis using the stiffness method. While it is convenient to have Python code embedded in the Grasshopper definition, as we will be using additional Python packages, including  NumPy and  SciPy, we need to take care to make sure these, as well as Python are loaded into Rhino and Grasshopper when we first run the software.

The most effective way of doing this when you first open Rhino 8 (version 8 is the minimum version that includes the required Python component) on a machine, before you open Grasshopper is to:

- From the menu bar in Rhino select Tools → Script → Edit... which will open the Script Editor in Rhino.
- In the Script Editor select File → New →  New Python 3 which will open a new Python script.
- In the Script Editor select the Install a third-party package for this script icon which will open the Install Package pop-up.
- In the Specify Packages(s) text entry type numpy, scipy and click Install.

Once you have done this you can open Grasshopper from Rhino. When the Grasshopper canvas opens it is strongly recommended that you select the following options, as these are the options that are used in the video tutorials:

- From the menu bar in Grasshopper select Display → Draw Icons.
- Select Display → Draw Fancy Wires.
- Select Display → Canvas Widgets → Profiler.
- Select View → Obscure Components.

Further options that are useful to be aware of when working in Grasshopper are:

- Solution → Disable Solver which can be useful if Grasshopper crashes and you want to re-open a definition.
- Solution → Recompute which can be used if a definition does not update following a change for example to a Python script.

A component that is not included by default in Grasshopper that is useful is  Bifocals which adds component names as labels.

A key combination that is useful if you are exploring a Grasshopper definition is `ctrl+alt` which brings up an  icon beside the  cursor and when used with `mouse-left-click` on a component shows you where the component can be found in Grasshopper.

There are many other useful features in Grasshopper that you will discover as you work through the video tutorials.

3.1 Python component in Grasshopper

The  Python 3 Script component is found in the Script collection of components on the Maths tab in Grasshopper. Like many Grasshopper components if you zoom in to it on the canvas you will notice  and  symbols on it which can be used to add or remove inputs and outputs, as shown in Figure 3.1.

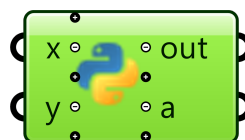


Figure 3.1:  Python 3 component in Grasshopper.

If you hover over one of the inputs on the left-hand-side of the component and mouse-right-click a menu will be brought up in which you need to decide whether the input requires Item Access, List Access, or Tree Access, based on its **data structure** and what **data type** the input is from the options under Type Hints. If you hover over one of the outputs on the right-hand-side of the component and mouse-right-click a menu will be brought up in which you need to decide what **data type** the output is under Type Hints, as shown in Figure 3.2

If you mouse-double-left-click on the component you will open the Script Editor as shown in Figure 3.3 in which you can write your Python script.

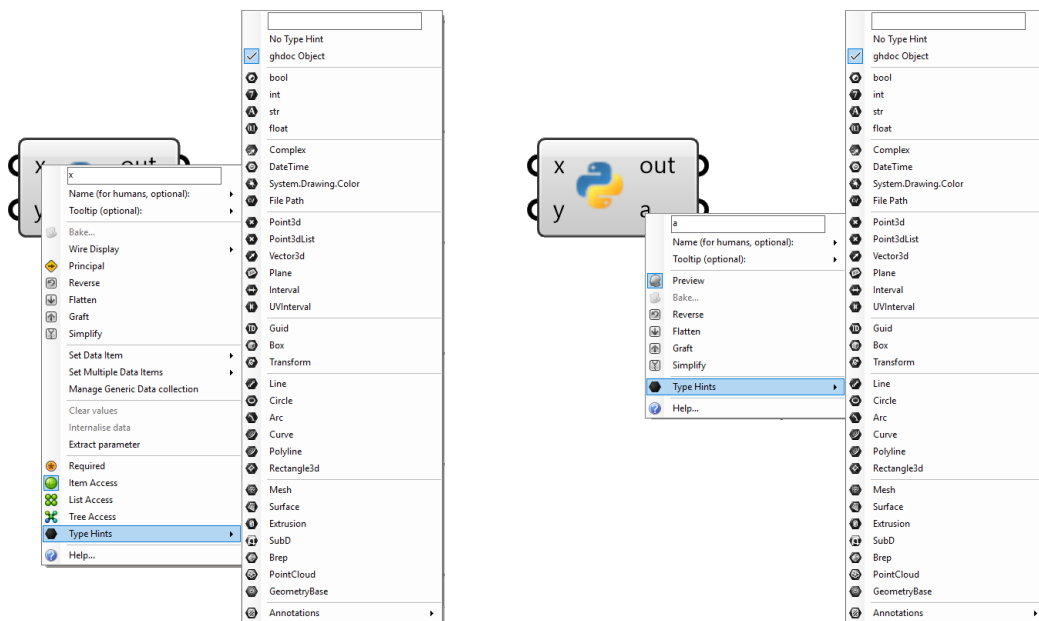


Figure 3.2: Input and output menus for the Python 3 component.

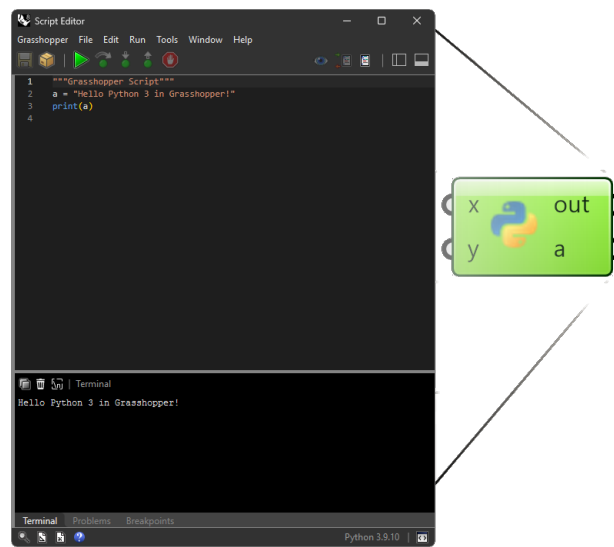


Figure 3.3: Script Editor opened from the Python 3 component.

From here you should work through the video tutorials to explore how to develop the pre-processing, structural analysis, and post-processing component groups of a Grasshopper definition.

A sample Python script which implements the stiffness method using Euler-Bernoulli beam elements in 2D, which has been tested in Grasshopper is attached to the notes.

 Python script implementing the stiffness method in Grasshopper.

4 Structural Analysis | Part 2

Having developed our initial structural analysis setup in Grasshopper and Python there are some further aspects that we can consider.

4.1 Equivalent nodal actions

We will consider cases where a uniformly distributed load (UDL) is applied along the length of an element in either the local or global axis systems.

4.1.1 UDLs applied in the local axis system

Considering a uniformly distributed load applied in the local x axis direction (w_x), as shown in Figure 4.1, we can calculate equivalent nodal actions in the global axis system as:

$$f_{\text{ena}} = w_x N^T \begin{bmatrix} L/2 \\ 0 \\ 0 \\ L/2 \\ 0 \\ 0 \end{bmatrix} \quad (4.1)$$

where f_{ena} is a 6×1 equivalent nodal action matrix, N^T is the transpose of the direction cosine matrix, and noting that w_x is taken as positive in the positive local x direction.

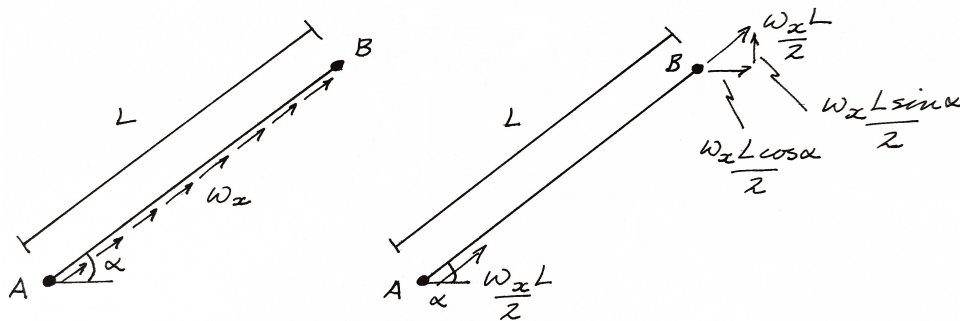


Figure 4.1: Equivalent nodal actions for a UDL applied in the local x direction.

Considering a uniformly distributed load applied in the local y axis direction (w_y), as shown in Figure 4.2, we can calculate equivalent nodal actions in the global axis system as:

$$f_{\text{ena}} = w_y N^T \begin{bmatrix} 0 \\ L/2 \\ L^2/12 \\ 0 \\ L/2 \\ -L^2/12 \end{bmatrix} \quad (4.2)$$

noting that w_y is taken as positive in the positive local y direction.

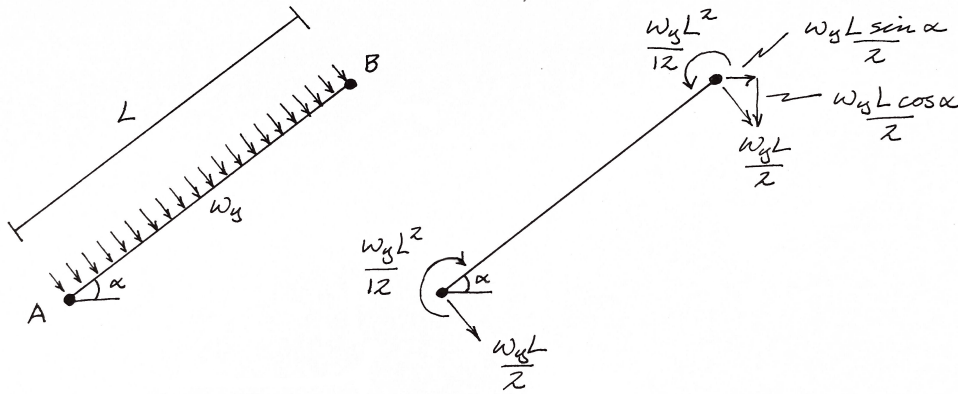


Figure 4.2: Equivalent nodal actions for a UDL applied in the local y direction.

Hence we can calculate the equivalent nodal actions in the global axis system with UDLs acting in both the local x and y directions as:

$$f_{\text{ena}} = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 & 0 & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & 0 & 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} w_x L/2 \\ w_y L/2 \\ w_y L^2/12 \\ w_x L/2 \\ w_y L/2 \\ -w_y L^2/12 \end{bmatrix} \quad (4.3)$$

4.1.2 UDLs applied in the global axis system

Considering UDLs applied in either the global x' or y' directions, we can transform these into the local axis system prior to calculating the equivalent nodal actions in the global axis system,

as shown in Figure 4.3. This can be done using the relationship:

$$\begin{bmatrix} w_x \\ w_y \end{bmatrix} = \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} w_{x'} \\ w_{y'} \end{bmatrix} \quad (4.4)$$

noting that the transformation is done using the basic unit of the direction cosine matrix.

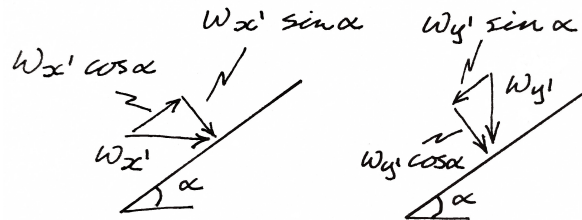


Figure 4.3: Transforming UDLs in the global axis system to the local axis system.

When UDLs are specified in the global axis system we need to note whether the element length L should be used to calculate the equivalent nodal actions, or whether the projected element lengths ($L_{x'}$ or $L_{y'}$) should be used. If the projected element length should be used then an adjustment based on the element angle can be made to the UDL magnitude.

As equivalent nodal actions are used to replicate the effect of UDLs in the force matrix (f) and do not represent applied forces at the nodes, they should be subtracted when calculating beam end forces or reactions.

4.1.3 Implementation in Grasshopper and Python

You should work through the video tutorials to explore how equivalent nodal actions can be included in a structural analysis Grasshopper definition.

Sample Python scripts which calculate equivalent nodal actions from local and global UDLs, and update the implementation of the stiffness method using Euler-Bernoulli beam elements in 2D to include equivalent nodal actions, are attached to the notes:

📎 Python script calculating equivalent nodal actions in Grasshopper.

📎 Updated Python script implementing the stiffness method in Grasshopper.

4.2 Internal pins

Internal pins, which allow free rotation, and result in a bending moment of zero at the pin, can be introduced into a structure by changing the element stiffness matrix for any element which connects to the node that the pin is at.

We will consider three different scenarios; a pin at the start node, a pin at the end node, and a pin at both the start and end nodes of an element.

4.2.1 Pin at the start node

If we have a pin (or rotational release) at the start node (node A) of an element then as $M_A = 0$, from the original element stiffness matrix (Equation 2.2) we can state:

$$0 = \frac{4EI}{L}\theta_A + \begin{bmatrix} 0 & \frac{6EI}{L^2} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \delta_{xB} \\ \delta_{yB} \\ \theta_B \end{bmatrix} \quad (4.5)$$

Hence θ_A is given as:

$$\theta_A = -\frac{L}{4EI} \begin{bmatrix} 0 & \frac{6EI}{L^2} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \delta_{xB} \\ \delta_{yB} \\ \theta_B \end{bmatrix} \quad (4.6)$$

Which can be rewritten as:

$$\theta_A = \begin{bmatrix} 0 & -\frac{3}{2L} & 0 & \frac{3}{2L} & -\frac{1}{2} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \delta_{xB} \\ \delta_{yB} \\ \theta_B \end{bmatrix} \quad (4.7)$$

We can restate the relationships given by the element stiffness matrix as:

$$\begin{bmatrix} R_{xA} \\ R_{yA} \\ R_{xB} \\ R_{yB} \\ M_B \end{bmatrix} = \begin{bmatrix} \frac{EA}{L} & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ -\frac{EA}{L} & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \delta_{xB} \\ \delta_{yB} \\ \theta_B \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{6EI}{L^2} \\ 0 \\ -\frac{6EI}{L^2} \\ \frac{2EI}{L} \end{bmatrix} \theta_A \quad (4.8)$$

Substituting in the relationship for θ_A we can find the stiffness matrix for an element with a pin at node A as:

$$k_{\text{pinA}} = \begin{bmatrix} \frac{EA}{L} & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ -\frac{EA}{L} & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -\frac{9EI}{L^3} & 0 & \frac{9EI}{L^3} & -\frac{3EI}{L^2} \\ 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{9EI}{L^3} & 0 & -\frac{9EI}{L^3} & \frac{3EI}{L^2} \\ 0 & -\frac{3EI}{L^2} & 0 & \frac{3EI}{L^2} & -\frac{EI}{L} \end{bmatrix} \quad (4.9)$$

As it is desirable to maintain a 6×6 element stiffness matrix we can restate the matrix for our Python code as:

$$k_{\text{pinA}} = \left[\begin{array}{ccc|ccc} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{3EI}{L^3} & 0 & 0 & -\frac{3EI}{L^3} & \frac{3EI}{L^2} \\ 0 & 0 & k_e & 0 & 0 & 0 \\ \hline -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{3EI}{L^3} & 0 & 0 & \frac{3EI}{L^3} & -\frac{3EI}{L^2} \\ 0 & \frac{3EI}{L^2} & 0 & 0 & -\frac{3EI}{L^2} & \frac{3EI}{L} \end{array} \right] \quad (4.10)$$

where k_e is an extremely small but non-zero stiffness required to prevent the matrix from being singular, which would prevent the inverse from being found, and prevent the use of the solve function from the linear algebra functions in the SciPy package in Python.

If a UDL is applied to an element with a pin at the start node we also need to adjust the equivalent nodal actions for the start and end nodes of the element. Considering a propped cantilever with the fixed end at node B and the prop at node A, for an element with a pin at node A, the equivalent nodal actions in the global axis system for a UDL applied in the local axis system are given as:

$$f_{\text{ena.pinA}} = N^T \begin{bmatrix} w_x L/2 \\ 3w_y L/8 \\ 0 \\ w_x L/2 \\ 5w_y L/8 \\ -w_y L^2/8 \end{bmatrix} \quad (4.11)$$

4.2.2 Pin at the end node

Similarly, if we have a pin (or rotational release) at the end node (node B) of an element then as $M_B = 0$, from the original element stiffness matrix (Equation 2.2) we can state:

$$0 = \frac{4EI}{L}\theta_B + \begin{bmatrix} 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \theta_A \\ \delta_{xB} \\ \delta_{yB} \end{bmatrix} \quad (4.12)$$

Hence θ_B is given as:

$$\theta_B = -\frac{L}{4EI} \begin{bmatrix} 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \theta_A \\ \delta_{xB} \\ \delta_{yB} \end{bmatrix} \quad (4.13)$$

Which can be rewritten as:

$$\theta_B = \begin{bmatrix} 0 & -\frac{3}{2L} & -\frac{1}{2} & 0 & \frac{3}{2L} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \theta_A \\ \delta_{xB} \\ \delta_{yB} \end{bmatrix} \quad (4.14)$$

We can restate the relationships given by the element stiffness matrix as:

$$\begin{bmatrix} R_{xA} \\ R_{yA} \\ M_A \\ R_{xB} \\ R_{yB} \end{bmatrix} = \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} \\ \hline -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \theta_A \\ \delta_{xB} \\ \delta_{yB} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{6EI}{L^2} \\ \frac{2EI}{L} \\ 0 \\ -\frac{6EI}{L^2} \end{bmatrix} \theta_B \quad (4.15)$$

Substituting in the relationship for θ_B we can find the stiffness matrix for an element with a pin at node B as:

$$k_{\text{pinB}} = \left[\begin{array}{ccc|cc} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} \\ \hline -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} \end{array} \right] + \left[\begin{array}{ccc|cc} 0 & 0 & 0 & 0 & 0 \\ 0 & -\frac{9EI}{L^3} & -\frac{3EI}{L^2} & 0 & \frac{9EI}{L^3} \\ 0 & -\frac{3EI}{L^2} & -\frac{EI}{L} & 0 & \frac{3EI}{L^2} \\ \hline 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{9EI}{L^3} & \frac{3EI}{L^2} & 0 & -\frac{9EI}{L^3} \end{array} \right] \quad (4.16)$$

As before it is desirable to maintain a 6×6 element stiffness matrix we can restate the matrix for our Python code as:

$$k_{\text{pinB}} = \left[\begin{array}{ccc|ccc} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{3EI}{L^3} & \frac{3EI}{L^2} & 0 & -\frac{3EI}{L^3} & 0 \\ 0 & \frac{3EI}{L^2} & \frac{3EI}{L} & 0 & -\frac{3EI}{L^2} & 0 \\ \hline -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{3EI}{L^3} & -\frac{3EI}{L^2} & 0 & \frac{3EI}{L^3} & 0 \\ 0 & 0 & 0 & 0 & 0 & k_e \end{array} \right] \quad (4.17)$$

where k_e is an extremely small but non-zero stiffness required to prevent the matrix from being singular.

If a UDL is applied to an element with a pin at the end node we also need to adjust the equivalent nodal actions for the start and end nodes of the element. Considering a propped cantilever with the fixed end at node A and the prop at node B, for an element with a pin at node B, the equivalent nodal actions in the global axis system for a UDL applied in the local axis system are given as:

$$f_{\text{ena.pinB}} = N^T \begin{bmatrix} w_x L/2 \\ 5w_y L/8 \\ w_y L^2/8 \\ w_x L/2 \\ 3w_y L/8 \\ 0 \end{bmatrix} \quad (4.18)$$

4.2.3 Pins at the start and end nodes

If we have pins at the start and end nodes of an element this is equivalent to having a bar element, which only resists axial load, as the bending moment is zero at both nodes. While this could be represented by only leaving the axial terms in the element stiffness matrix we will leave in the terms linking displacements and forces in the y direction in the local axis system to provide stiffness in this direction.

$$k_{\text{pinAB}} = \left[\begin{array}{ccc|ccc} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{3EI}{L^3} & 0 & 0 & -\frac{3EI}{L^3} & 0 \\ 0 & 0 & k_e & 0 & 0 & 0 \\ \hline -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{3EI}{L^3} & 0 & 0 & \frac{3EI}{L^3} & 0 \\ 0 & 0 & 0 & 0 & 0 & k_e \end{array} \right] \quad (4.19)$$

where k_e is an extremely small but non-zero stiffness required to prevent the matrix from being singular.

If a UDL is applied to an element with a pins at the start and end nodes we also need to adjust the equivalent nodal actions for the start and end nodes of the element. Considering a beam with a bending moment of zero at each node the equivalent nodal actions in the global axis system for a UDL applied in the local axis system are given as:

$$f_{\text{ena.pinAB}} = N^T \begin{bmatrix} w_x L/2 \\ w_y L/2 \\ 0 \\ w_x L/2 \\ w_y L/2 \\ 0 \end{bmatrix} \quad (4.20)$$

4.2.4 Implementation in Grasshopper and Python

You should work through the video tutorials to explore how pins can be included in a structural analysis Grasshopper definition.

Sample Python scripts which update the implementation of the stiffness method using Euler-Bernoulli beam elements in 2D to include internal pins, and which calculates equivalent nodal actions from local and global UDLs, including adjustments required when internal pins are introduced into the structure, are attached to the notes:

-  Updated Python script implementing the stiffness method in Grasshopper.
-  Updated python script calculating equivalent nodal actions in Grasshopper.

5 Eigenanalysis

Eigenanalysis is a linear algebra method that can be applied to square matrices, finding eigenvalues and eigenvectors for that matrix. When eigenvectors are multiplied by the original matrix this gives the same result as scaling the eigenvector by a constant eigenvalue, with corresponding eigenvalues and eigenvectors referred to as eigenpairs. The prefix 'eigen' is German and can be taken to mean 'own' or 'characteristic'. The relationship can be stated:

$$(A - \lambda I)\phi = 0 \quad (5.1)$$

where A is a square ($n \times n$) matrix, I is the identity matrix, λ is an eigenvalue, and ϕ is an eigenvector matrix. The relationship can also be stated:

$$A\phi = \lambda\phi \quad (5.2)$$

Eigenanalysis can be used to find principal stresses, strains, and second moments of area, values (eigenvalues) and directions (eigenvectors).

5.1 Eigenanalysis using the stiffness matrix

When carrying out structural analysis we can perform eigenanalysis on the system stiffness matrix (K) to find principal displacement modes (eigenvectors or eigenmodes) and their associated stiffness values (eigenvalues). This can be useful in checking that structures do not have unpredicted low stiffness displacement modes that have not been considered in their design. A zero eigenvalue indicates the structure can either undergo rigid body motion or that it contains a mechanism. Eigenanalysis can be used with sensor data from structural health monitoring to assess how a built structure behaves compared to structural analysis predictions, while a change in eigenvalues or eigenvectors may be associated with degradation of a structure.

In Python we can use the `eig` function available from the linear algebra functions in the SciPy package (with `eig(K)`) to find the eigenvalues and eigenvectors for the relationship:

$$K\phi = \lambda\phi \quad (5.3)$$

For a K matrix with dimensions $n \times n$ the eigenanalysis will result in n eigenvalues and n eigenvectors, each given as a column in an $n \times n$ matrix, with eigenpairs given as the i^{th} eigenvalue and the i^{th} column in the eigenvector matrix. Usually the largest eigenvalues will be associated with eigenvectors where displacement takes place at the degrees of freedom that have constrained boundary conditions, while the lowest eigenvalues will be associated with eigenvectors describing displacement modes (shapes) that may be potential buckling modes.

5.2 Eigenanalysis using the geometric stiffness matrix

While eigenanalysis of the K matrix gives eigenvectors that may represent potential buckling mode shapes, the corresponding eigenvalues do not give buckling loads. To get buckling loads and buckling modes we need to use the geometric stiffness matrix (K_g) which depends on the system stiffness matrix (K) and the applied loading. As we are primarily concerned with the application of the stiffness method, we will not derive the geometric stiffness matrix here, but will comment that it results in the stiffness of elements in tension increasing, while the stiffness of elements in compression decreases. The geometric stiffness matrix for a 2D beam element is given as:

$$k_g = \frac{F_x}{30L} \left[\begin{array}{ccc|ccc} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 36 & 3L & 0 & -36 & 3L \\ 0 & 3L & 4L^2 & 0 & -3L & -L^2 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -36 & -3L & 0 & 36 & -3L \\ 0 & 3L & -L^2 & 0 & -3L & 4L^2 \end{array} \right] \quad (5.4)$$

where F_x is the axial force calculated based on the change in length of the element:

$$F_x = \Delta L \left(\frac{EA}{L} \right) \quad (5.5)$$

where ΔL is the change in length, with tension (extension) positive and compression (shortening) negative.

As the K_g matrix is dependent on the applied loading, the resulting eigenvalues represent multiples of the applied loading. In Python we can use the `eig` function available from the linear algebra functions in the SciPy package (with `eig(K, -Kg)`) to find the eigenvalues and

eigenvectors for the relationship:

$$K\phi = -\lambda K_g\phi \quad (5.6)$$

An eigenvalue < 1 indicates that the structure will buckle with the buckling mode shape given by the associated eigenvector at a lower load than the applied load in the analysis, while eigenvalues ≥ 1 give the ratio the applied load can be increased by before buckling is predicted to occur. An eigenvalue < 0 indicates buckling is predicted to occur with a reversal of the applied load. Buckling mode shapes are often used as imperfections in geometric non-linear analysis. Depending on the results of a buckling eigenanalysis, a structure may be altered to preference a buckling mode shape (eigenvector) with a higher buckling load (eigenvalue) if this prevents a lower buckling mode shape from occurring. The geometric stiffness matrix (K_g) is also used in geometric non-linear analysis.

Solutions found for buckling loads using eigenanalysis for a column may be compared to those found using Euler's buckling theory, stated as:

$$P_{cr,n} = \frac{n^2\pi^2 EI}{L_e^2} \quad (5.7)$$

where $P_{cr,n}$ is the buckling load for the n^{th} buckling mode, n is an integer value, and L_e is the effective length, which for a pin-pin ended column is L .

5.2.1 Adjusting the geometric stiffness matrix for an internal pin

We can adjust the element geometric stiffness matrix (k_g) if an internal pin is introduced using a similar process as we used to adjust the element stiffness matrix (k). We will consider three different scenarios; a pin at the start node, a pin at the end node, and pins at both the start and end nodes of an element.

For an internal pin at the start node (node A) of an element then as $M_A = 0$, from the original element geometric stiffness matrix (Equation 5.4) we can state:

$$\theta_A = \begin{bmatrix} 0 & -\frac{3}{4L} & 0 & \frac{3}{4L} & \frac{1}{4} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \delta_{xB} \\ \delta_{yB} \\ \theta_B \end{bmatrix} \quad (5.8)$$

Substituting in the relationship for θ_A we can find the geometric stiffness matrix for a element with a pin at node A as:

$$k_{g(\text{pinA})} = \frac{F_x}{30L} \left[\begin{array}{cc|ccc} 0 & 0 & 0 & 0 & 0 \\ 0 & 36 & 0 & -36 & 3L \\ \hline 0 & 0 & 0 & 0 & 0 \\ 0 & -36 & 0 & 36 & -3L \\ 0 & 3L & 0 & -3L & 4L^2 \end{array} \right] + \frac{F_x}{40L} \left[\begin{array}{cc|ccc} 0 & 0 & 0 & 0 & 0 \\ 0 & -3 & 0 & 3 & L \\ \hline 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & -3 & -L \\ 0 & L & 0 & -L & -L^2/3 \end{array} \right] \quad (5.9)$$

As it is desirable to maintain a 6×6 element matrix we can restate the matrix as:

$$k_{g(\text{pinA})} = \frac{F_x}{40L} \left[\begin{array}{ccc|ccc} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 45 & 0 & 0 & -45 & 5L \\ 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -45 & 0 & 0 & 45 & -5L \\ 0 & 5L & 0 & 0 & -5L & 5L^2 \end{array} \right] \quad (5.10)$$

Similarly. for an internal pin at the end node (node B) of an element then as $M_B = 0$, from the original element geometric stiffness matrix (Equation 5.4) we can state:

$$\theta_B = \begin{bmatrix} 0 & -\frac{3}{4L} & \frac{1}{4} & 0 & \frac{3}{4L} \end{bmatrix} \begin{bmatrix} \delta_{xA} \\ \delta_{yA} \\ \theta_A \\ \delta_{xB} \\ \delta_{yB} \end{bmatrix} \quad (5.11)$$

Substituting in the relationship for θ_B we can find the geometric stiffness matrix for a

element with a pin at node B as:

$$k_{g(\text{pinB})} = \frac{F_x}{30L} \left[\begin{array}{ccc|cc} 0 & 0 & 0 & 0 & 0 \\ 0 & 36 & 3L & 0 & -36 \\ 0 & 3L & 4L^2 & 0 & -3L \\ \hline 0 & 0 & 0 & 0 & 0 \\ 0 & -36 & -3L & 0 & 36 \end{array} \right] + \frac{F_x}{40L} \left[\begin{array}{ccc|cc} 0 & 0 & 0 & 0 & 0 \\ 0 & -3 & L & 0 & 3 \\ 0 & L & -L^2/3 & 0 & -L \\ \hline 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & -L & 0 & -3 \end{array} \right] \quad (5.12)$$

As it is desirable to maintain a 6×6 element matrix we can restate the matrix as:

$$k_{g(\text{pinB})} = \frac{F_x}{40L} \left[\begin{array}{ccc|ccc} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 45 & 5L & 0 & -45 & 0 \\ 0 & 5L & 5L^2 & 0 & -5L & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -45 & -5L & 0 & 45 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \quad (5.13)$$

For internal pins at the start and end nodes (nodes A and B) the adjusted element geometric matrix is given:

$$k_{g(\text{pinAB})} = \frac{F_x}{40L} \left[\begin{array}{ccc|ccc} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 45 & 0 & 0 & -45 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -45 & 0 & 0 & 45 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \quad (5.14)$$

5.3 Eigenanalysis using the mass matrix

We can also use eigenanalysis to predict natural frequencies (eigenvalues) and their associated vibration mode shapes (eigenvectors) using the mass matrix (M). For a 2D beam element we can define either a distributed mass matrix, which takes account of how mass is distributed along the length of an element, or we can use a lumped mass matrix, which assumes that the

mass is lumped at each node of the element.

The distributed mass matrix for a 2D beam is given as:

$$m = \frac{\rho AL}{420} \left[\begin{array}{ccc|ccc} 140 & 0 & 0 & 70 & 0 & 0 \\ 0 & 156 & 22L & 0 & 54 & -13L \\ 0 & 22L & 4L^2 & 0 & 13L & -3L^2 \\ \hline 70 & 0 & 0 & 140 & 0 & 0 \\ 0 & 54 & 13L & 0 & 156 & -22L \\ 0 & -13L & -3L^2 & 0 & -22L & 4L^2 \end{array} \right] \quad (5.15)$$

where ρ is the density of the material.

The lumped mass matrix for a 2D beam is given as:

$$m = \frac{\rho AL}{2} \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \quad (5.16)$$

Until now our analyses have been unit agnostic, in that the units of our output depend on the units of our input. When finding natural frequencies we must adopt a unit for density that is consistent with the definition of a Newton as **1N = 1kg.m/s²** where the base SI units (from the French système international d'unités) are Newtons (N) for force, kilograms (kg) for mass, metres (m) for length, and seconds (s) for time, with acceleration in m/s². Combinations of SI units that satisfy the definition of a Newton are given in Table 2 where 1tonne = 1000kg.

Force	Length	Mass	Density	Time
N	m	kg	kg/m ³	s
kN	m	tonne	tonne/m ³	s
N	mm	tonne	tonne/mm ³	s

Table 2: Consistent SI unit combinations.

The M matrix is independent of the applied loading. In Python we can use the eig

function available from the linear algebra functions in the SciPy package (with `eig(K,M)`) to find the eigenvalues and eigenvectors for the relationship:

$$K\phi = \lambda M\phi = \omega^2 M\phi \quad (5.17)$$

where ω is the angular frequency with units of radians per time (usually seconds), which gives the frequency as a function of the eigenvalue:

$$f = \frac{\omega}{2\pi} = \frac{\sqrt{\lambda}}{2\pi} \quad (5.18)$$

where f is the frequency with units of cycles per time (usually seconds), in Hz (hertz).

It can be useful to find the natural frequencies of a structure to check whether these correspond to likely excitation frequencies within the loading that they are subjected to, for example from pedestrians, crowd movement, vortex shedding, or in an earthquake. Additional lumped masses and tuned mass dampers may be added to alter the response of the structure. The mass matrix (M) is also used in dynamic analysis.

Solutions found for natural frequencies using eigenanalysis for a simply supported beam may be compared to those found using the formula:

$$f_n = \frac{n^2\pi}{2L^2} \sqrt{\frac{EI}{\rho A}} \quad (5.19)$$

where f_n is the n^{th} natural frequency, and n is an integer value.

Solutions found for natural frequencies using eigenanalysis for a cantilever may be compared to those found using the formula:

$$f_n = \frac{(\beta_n L)^2}{2\pi L^2} \sqrt{\frac{EI}{\rho A}} \quad (5.20)$$

where f_n is the n^{th} natural frequency, n is an integer value, and $\beta_n L$ are constants that have the values 1.875, 4.694, 7.855, 10.996, and 14.137 for the first five vibration modes for a cantilever beam. For higher modes from the fourth mode $\beta_n L$ can be approximated as:

$$\beta_n L \approx \frac{(2n-1)\pi}{2} \quad (5.21)$$

5.3.1 Adjusting the mass matrix for an internal pin

We can adjust the element mass matrix (m) if an internal pin is introduced. We will consider three different scenarios; a pin at the start node, a pin at the end node, and pins at both the start and end nodes of an element. The adjustment can be carried out in a similar way to the adjustment for the element stiffness matrix (k) based on substitution, often referred to as Guyan reduction¹.

For an internal pin at the start node (node A) we can define a transformation matrix using the identity matrix I and the substitution terms from Equation 4.7:

$$S_{\text{pinA}} = \begin{bmatrix} 0 & -\frac{3}{2L} & 0 & \frac{3}{2L} & -\frac{1}{2} \end{bmatrix} \quad (5.22)$$

Giving the transformation matrix as:

$$T_{\text{pinA}} = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -\frac{3}{2L} & 0 & 0 & \frac{3}{2L} & -\frac{1}{2} \\ \hline 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right] \quad (5.23)$$

The adjusted element mass matrix can then be derived as:

¹Guyan, R.J., Reduction of stiffness and mass matrices, AIAA Journal, 1965, doi:10.2514/3.2874

$$m_{\text{pinA}} = T_{\text{pinA}}^T m T_{\text{pinA}} = \frac{\rho AL}{840} \left[\begin{array}{ccc|ccc} 280 & 0 & 0 & 140 & 0 & 0 \\ 0 & 198 & 0 & 0 & 117 & -33L \\ 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 140 & 0 & 0 & 280 & 0 & 0 \\ 0 & 117 & 0 & 0 & 408 & -72L \\ 0 & -33L & 0 & 0 & -72L & 16L^2 \end{array} \right] \quad (5.24)$$

Similarly, for an internal pin at the end node (node B) we can define a transformation matrix using the identify matrix I and the substitution terms from Equation 4.14:

$$S_{\text{pinB}} = \left[\begin{array}{cccc} 0 & -\frac{3}{2L} & -\frac{1}{2} & 0 & \frac{3}{2L} \end{array} \right] \quad (5.25)$$

Giving the transformation matrix as:

$$T_{\text{pinB}} = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & -\frac{3}{2L} & -\frac{1}{2} & 0 & \frac{3}{2L} & 0 \end{array} \right] \quad (5.26)$$

The adjusted element mass matrix can then be derived as:

$$m_{\text{pinB}} = T_{\text{pinB}}^T m T_{\text{pinB}} = \frac{\rho AL}{840} \left[\begin{array}{ccc|ccc} 280 & 0 & 0 & 140 & 0 & 0 \\ 0 & 408 & 72L & 0 & 117 & 0 \\ 0 & 72L & 16L^2 & 0 & 33L & 0 \\ \hline 140 & 0 & 0 & 280 & 0 & 0 \\ 0 & 117 & 33L & 0 & 198 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \quad (5.27)$$

For internal pins at the start and end nodes (nodes A and B) the adjusted element mass matrix is given:

$$m_{\text{pinAB}} = \frac{\rho AL}{6} \left[\begin{array}{ccc|ccc} 2 & 0 & 0 & 1 & 0 & 0 \\ 0 & 2 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & 2 & 0 & 0 \\ 0 & 1 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \quad (5.28)$$

5.4 Implementation in Grasshopper and Python

In Grasshopper we can plot the eigenvectors using a normalised scale, where we set the amplitude of the mode shape given by the eigenvector to 1 and apply a magnification factor based on the overall dimensions of the structure.

You should work through the video tutorials to explore how eigenanalysis can be included in a structural analysis Grasshopper definition.

Sample Python scripts are included for eigenanalysis using the K matrix, constructing the K_g matrix and eigenanalysis using the K and K_g matrices, and constructing the distributed M matrix and eigenanalysis using the K and M matrices. The `eig` function available from the linear algebra functions in the SciPy package is used as this allows eigenvalues and eigenvectors to be found for what are referred to as left and right hand side matrices.

- 🔗 Python script implementing eigenanalysis using the K matrix.
- 🔗 Python script implementing eigenanalysis using the K and K_g matrices.
- 🔗 Eigenanalysis using the K and K_g matrices, including internal pins.
- 🔗 Python script implementing eigenanalysis using the K and M matrices.
- 🔗 Eigenanalysis using the K and M matrices, including internal pins.

Appendix A

Supplementary material

1 Structural finite element formulations

Please note that some nomenclature in the supplementary material may be inconsistent with that used in the main part of the notes.

1.1 Axial bar element

Let us first consider a bar element, that is an element which only resists axial forces. At this stage we will also introduce a local coordinate system onto the bar such that at the first node, n_1 we have $\xi = -1$, and at the second node, n_2 we have $\xi = +1$, as shown in Figure 1.1.

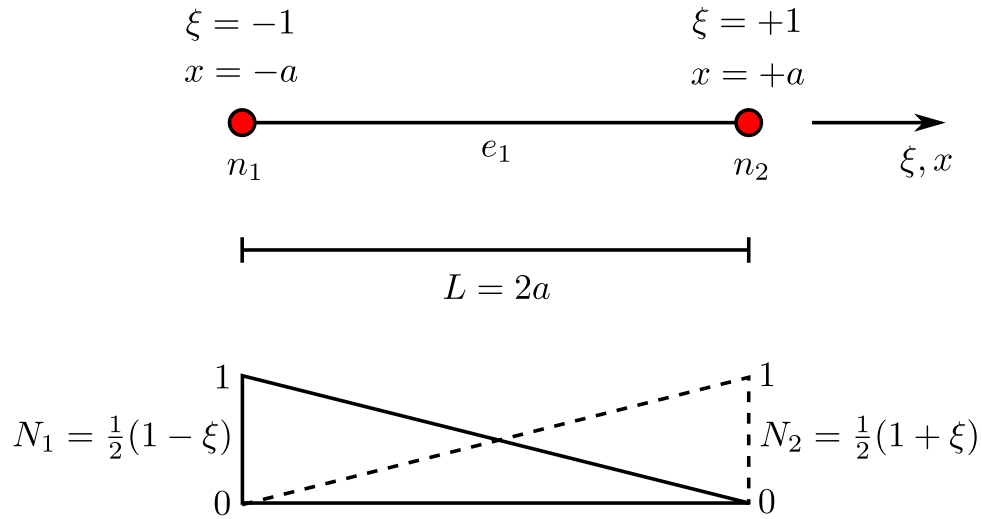


Figure 1.1: 1D bar element and associated linear shape functions

The total strain energy can be expressed as:

$$U = \frac{1}{2} \int_{-a}^{+a} \sigma_x \epsilon_x A \, dx \quad (1.1)$$

where A is the cross-sectional area of the bar (assumed to be constant). Taking the elastic modulus of the material to be E we have $\sigma_x = E\epsilon_x$ and:

$$U = \frac{1}{2} \int_{-a}^{+a} EA \epsilon_x^2 \, dx \quad (1.2)$$

with $\epsilon_x = \frac{du}{dx}$ we have:

$$U = \frac{1}{2} \int_{-a}^{+a} EA \left(\frac{du}{dx} \right)^2 dx \quad (1.3)$$

The external work associated with the axial deformation of the element is:

$$\partial W = \int_{-a}^{+a} p \partial u \, dx \quad (1.4)$$

In the relative coordinate system the axial displacement function can be expressed as:

$$u = N_1(\xi)u_1 + N_2(\xi)u_2 \quad (1.5)$$

which can be expressed in matrix form:

$$u = Nu_e = [N_1 \, N_2] \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \quad (1.6)$$

The strain energy in terms of the relative coordinate system can be expressed:

$$U = \frac{1}{2} \int_{-1}^{+1} EA \left(\frac{1}{a^2} \right) \left(\frac{du}{d\xi} \right)^2 a \, d\xi \quad (1.7)$$

$\frac{du}{d\xi}$ is found from $\frac{\partial}{\partial \xi} = J \frac{\partial}{\partial x}$ where J is referred to as the Jacobian operator, or derivative transformation matrix between the global and the local coordinate systems.

$$\frac{\partial}{\partial x} = J^{-1} \frac{\partial}{\partial \xi} \quad (1.8)$$

In the case of a bar element $J = \frac{\partial x}{\partial \xi} = a$, and $J^{-1} = \frac{1}{a}$. In addition it is noted that $x = a\xi$.

Given $\frac{du}{d\xi} = \left[\frac{dN_1}{d\xi} \, \frac{dN_2}{d\xi} \right] \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = N' u_e$ the strain energy can be expressed:

$$U = \frac{1}{2} u_e^T \int_{-1}^{+1} \frac{EA}{a} N'^T N' \, d\xi \, u_e \quad (1.9)$$

giving:

$$U = \frac{1}{2} u_e^T K_e u_e \quad (1.10)$$

where K_e is the element stiffness matrix:

$$K_e = \frac{EA}{a} \int_{-1}^{+1} N'^T N' d\xi \quad (1.11)$$

In analysis codes you will often find that the element stiffness matrix for a bar element is given:

$$K_e = B^T B W EA J^{-1} \quad (1.12)$$

where B is the derivative of the natural shape function N' in the ξ coordinate system, W is a Gauss point weighting, discussed later, and J^{-1} is the inverse of the Jacobian matrix.

For the bar element $\frac{dN_1}{d\xi} = -\frac{1}{2}$ and $\frac{dN_2}{d\xi} = +\frac{1}{2}$ and the element stiffness matrix is:

$$K_e = \frac{EA}{a} \int_{-1}^{+1} \begin{bmatrix} -\frac{1}{2} \\ +\frac{1}{2} \end{bmatrix} \begin{bmatrix} -\frac{1}{2} & +\frac{1}{2} \end{bmatrix} d\xi = \frac{EA}{2a} \begin{bmatrix} +1 & -1 \\ -1 & +1 \end{bmatrix} = \frac{EA}{L} \begin{bmatrix} +1 & -1 \\ -1 & +1 \end{bmatrix} \quad (1.13)$$

1.2 Euler-Bernoulli beam element

The Euler-Bernoulli beam element is based on standard or engineering beam bending theory. The assumptions used in formulating standard beam bending theory are:

- Plane sections remain plane after deformation.
- Plane sections remain perpendicular to the neutral axis after deformation.

These assumptions are illustrated in Figure 1.2.

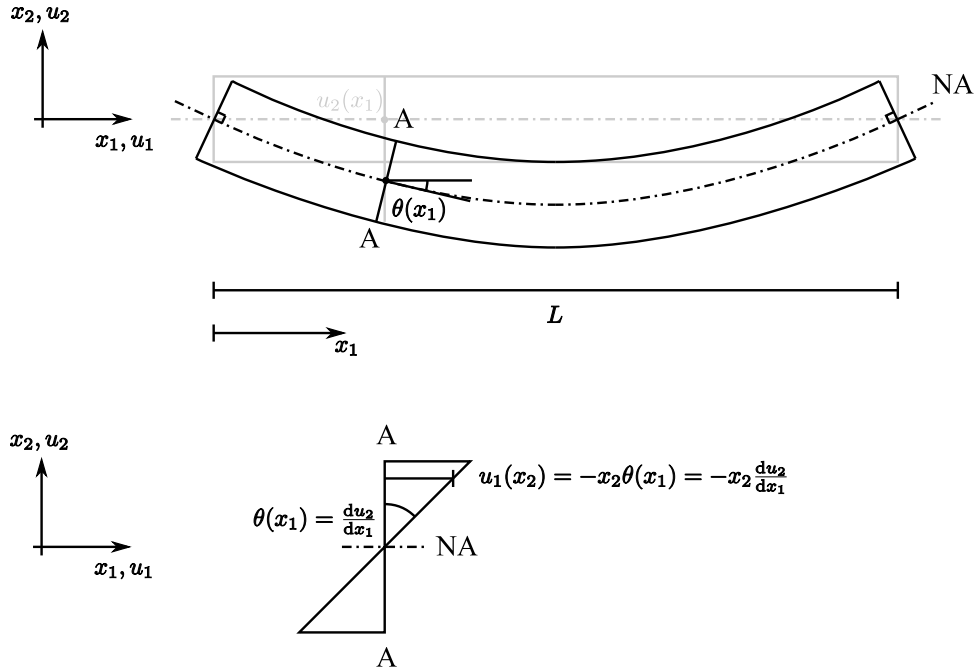


Figure 1.2: Euler-Bernoulli Beam Bending

Based on Figure 1.2 and the standard definition of strain we have the relationship:

$$\epsilon_{11}(x_2) = \frac{du_1}{dx_1} = \frac{d}{dx_1} \left(-x_2 \frac{du_2}{dx_1} \right) = -x_2 \frac{d^2u_2}{dx_1^2} = -x_2 \phi$$

Where ϕ is the curvature. The curvature of the beam is taken as:

$$\phi = \frac{1}{R} = \frac{d\theta}{ds} = \frac{d^2u_2}{dx_1^2}$$

Where R is the radius of curvature, and ds is the distance along the curve as illustrated in Figure 1.3, which for small deflections can be considered equal to dx_1 .

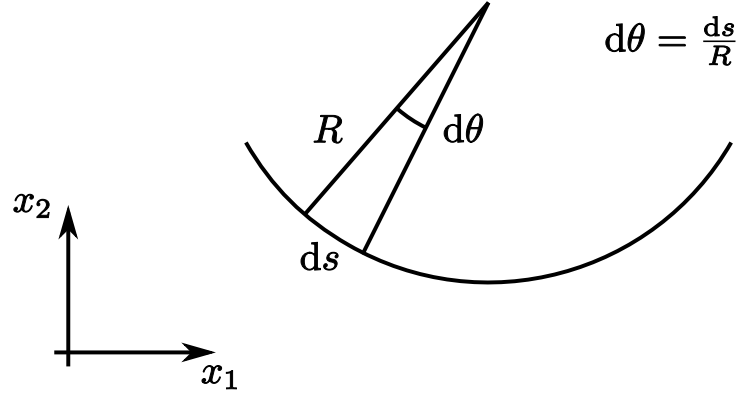


Figure 1.3: Beam curvature

With regards to a finite element formulation for a two noded element we can specify two degrees of freedom at each node, as shown in Figure 1.4.

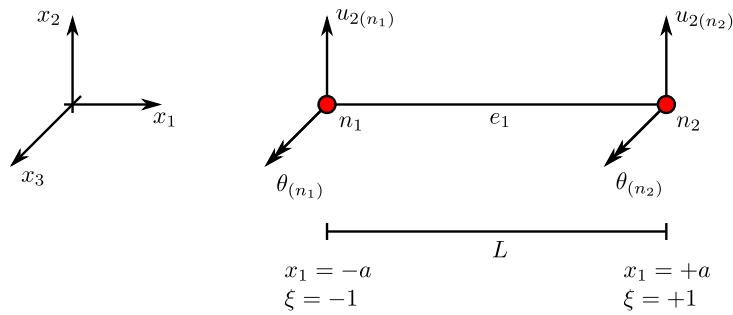


Figure 1.4: Degrees of freedom for a two noded Euler-Bernoulli beam element

We can define four shape functions as illustrated in Figure 1.5 and given:

$$N_1(\xi) = +\frac{1}{4}(1 - \xi)^2(2 + \xi) \quad (1.14)$$

$$N_2(\xi) = +\frac{1}{8}L(1 - \xi)^2(1 + \xi) \quad (1.15)$$

$$N_3(\xi) = +\frac{1}{4}(1 + \xi)^2(2 - \xi) \quad (1.16)$$

$$N_4(\xi) = -\frac{1}{8}L(1 + \xi)^2(1 - \xi) \quad (1.17)$$

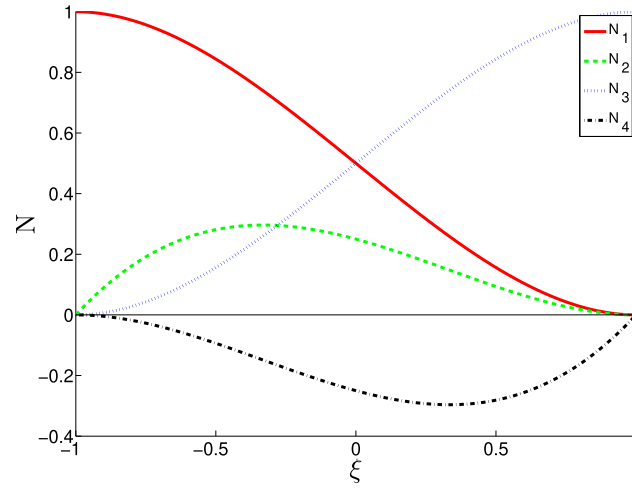


Figure 1.5: Hermite cubic shape functions for an Euler-Bernoulli beam element

The adopted Hermite cubic shape functions are said to have C^1 continuity, whereas our original linear shape functions had C^0 continuity. That is to say that C^1 shape functions are continuous across nodes for the first derivative, while C^0 shape functions are continuous for the value only. It is noted that $N_2(\xi)$ and $N_4(\xi)$ do not reach values of 1 or zero at the relevant nodes. However these shape functions relate to the angle of rotation, θ , or the slope at each of the nodes, $\theta = \frac{du_2}{dx_1}$.

Thus the relationship is true for $\frac{dN_2}{d\xi}$ and $\frac{dN_4}{d\xi}$ as shown in Figure 1.6 and given:

$$\frac{dN_2}{d\xi} = \frac{1}{4}(-1 - 2\xi + 3\xi^2) \quad (1.18)$$

$$\frac{dN_4}{d\xi} = \frac{1}{4}(-1 + 2\xi + 3\xi^2) \quad (1.19)$$

The strain energy associated with bending of the element can be obtained as:

$$U = \frac{1}{2} \int_V E \epsilon_{11}^2 dV \quad (1.20)$$

Where $dV = dA dx_1$, thus:

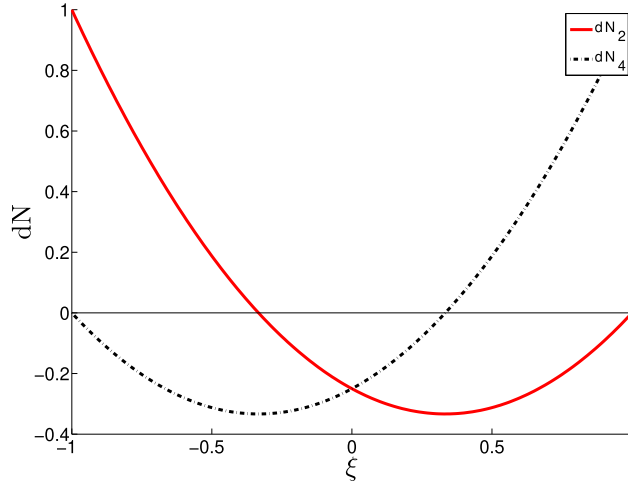


Figure 1.6: dN_2 and dN_4 for an Euler-Bernoulli beam element

$$U = \frac{1}{2} \int_{-a}^{+a} EI \left(\frac{\partial^2 u_2}{\partial x_1^2} \right)^2 dx_1 = \frac{1}{2} \int_{-1}^{+1} \frac{EI}{a^4} \left(\frac{\partial^2 u_2}{\partial \xi^2} \right)^2 a d\xi \quad (1.21)$$

From the chain rule it is found that $\frac{d^2 u_2}{d\xi^2} = \frac{1}{a^2} \frac{d^2 u_2}{dx_1^2} = \frac{4}{L^2} \frac{d^2 u_2}{dx_1^2}$. In addition it is noted that $x = a\xi = \frac{L}{2}\xi$.

Given $\frac{d^2 u_2}{d\xi^2} = \frac{d^2 N}{d\xi^2} u_e = N'' u_e$ the strain energy can be expressed:

$$U = \frac{1}{2} u_e^T \frac{EI}{a^3} \int_{-1}^{+1} N''^T N'' d\xi u_e \quad (1.22)$$

giving

$$U = \frac{1}{2} u_e^T K_e u_e \quad (1.23)$$

where K_e is the element stiffness matrix:

$$K_e = EI \int_{-1}^{+1} N''^T N'' a d\xi \quad (1.24)$$

and:

$$N'' = \frac{1}{L} \begin{bmatrix} 6\frac{\xi}{L} & 3\xi - 1 & -6\frac{\xi}{L} & 3\xi + 1 \end{bmatrix} \quad (1.25)$$

Using the Hermite shape functions the element stiffness matrix is found as:

$$K_e = \frac{EI}{L^3} \begin{bmatrix} 12 & 6L & -12 & 6L \\ 6L & 4L^2 & -6L & 2L^2 \\ -12 & -6L & 12 & -6L \\ 6L & 2L^2 & -6L & 4L^2 \end{bmatrix} \quad (1.26)$$

It is observed that this is the same as the matrix derived using a virtual work approach, therefore an exact match should be found between the stiffness method and the finite element method when using Euler-Bernoulli beam elements as formulated here.

1.3 Timoshenko beam element

Alternative beam bending theories to the standard beam bending theory exist, and the formulation of the Timoshenko beam element is an example of the application of one of these.

Timoshenko beam bending theory can be considered as an extension of standard beam bending theory, in which deflections due to shear strains are also taken into account. The assumptions used in formulating Timoshenko beam theory are:

- Plane sections remain in plane after deformation.
- Plane sections need not remain perpendicular to the neutral axis.

The first of these assumptions in particular will be looked at with regards to compatibility. The additional contribution to deformation due to considering shear strains is illustrated in Figure 1.7. The contribution of shear strains to deformation of a beam is seen to increase as the depth:span ratio increases.

It is seen in Figure 1.7 that the assumption of plane sections remaining in plane after deformation doesn't fit with what we know about the shear distribution within a section. This can to some extent be taken account of through the introduction of a shear correction coefficient, or shape factor, κ in our shear deformation relationship:

$$\gamma_{12} = \frac{V}{\kappa GA} \quad (1.27)$$

Where V is the shear force acting across the section in the x_2 direction, G is the shear modulus, normally taken as $E/2(1 + \nu)$ where ν is Poisson's ratio, and A is the area of the cross-section. Various values of κ have been put forward for cross-sections of different shapes, although $5/6$ is often used.

The finite element formulation for a Timoshenko beam element is different to the formulation for an Euler-Bernoulli beam element. Expressions for the curvature, ϕ , caused by bending moments, and the angle, γ , caused by shear forces can be written in matrix form as:

$$\begin{bmatrix} \phi \\ \gamma \end{bmatrix} = \begin{bmatrix} 0 & \frac{d}{dx_1} \\ -\frac{d}{dx_1} & 1 \end{bmatrix} \begin{bmatrix} u_2 \\ \theta \end{bmatrix} \quad (1.28)$$

48

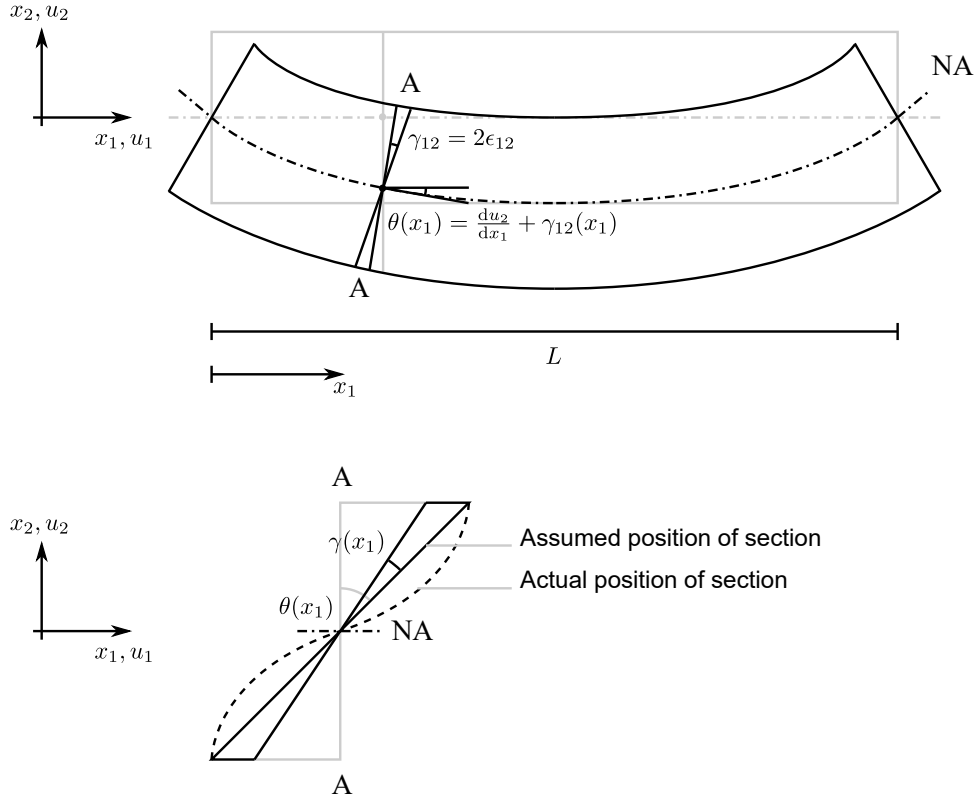


Figure 1.7: Timoshenko Beam Bending

Expressions for the bending moment, M , and the shear force, V can be written in matrix form as:

$$\begin{bmatrix} M \\ V \end{bmatrix} = \begin{bmatrix} EI & 0 \\ 0 & \kappa GA \end{bmatrix} \begin{bmatrix} \phi \\ \gamma \end{bmatrix} \quad (1.29)$$

We can define a Timoshenko beam element using the same degrees of freedom at each node as we had for the Euler-Bernoulli beam element, as shown in Figure 1.4. For the formulation presented here we will use linear shape functions.

$$\begin{bmatrix} u_2 \\ \theta \end{bmatrix} = \begin{bmatrix} \frac{1}{2}(1 - \xi) & 0 & \frac{1}{2}(1 + \xi) & 0 \\ 0 & \frac{1}{2}(1 - \xi) & 0 & \frac{1}{2}(1 + \xi) \end{bmatrix} \begin{bmatrix} u_{2(n_1)} \\ \theta_{(n_1)} \\ u_{2(n_2)} \\ \theta_{(n_2)} \end{bmatrix} \quad (1.30)$$

We can define the B matrix as:

$$B = dN = \begin{bmatrix} 0 & \frac{d}{dx_1} \\ -\frac{d}{dx_1} & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{2}(1-\xi) & 0 & \frac{1}{2}(1+\xi) & 0 \\ 0 & \frac{1}{2}(1-\xi) & 0 & \frac{1}{2}(1+\xi) \end{bmatrix} \quad (1.31)$$

The element stiffness matrix can then be defined as:

$$K_e = \int_V B^T D B \, dV \quad (1.32)$$

where $D = \begin{bmatrix} EI & 0 \\ 0 & \kappa GA \end{bmatrix}$

It is observed that the expression for K_e is a general relationship, such that we could have used it in the development of the axial bar element, and the Euler-Bernoulli beam element formulations. It is easy to implement using short blocks of code within a numerical coding language such as Python, and forms the basis of other structural and continuum finite element formulations.

1.4 Numerical integration

Integrals arising from the computational implementation of the finite element method may be solved using numerical integration, for example Gauss quadrature. Considering a function $f(\xi), \xi \in [-1, +1]$ the integral $\int_{-1}^{+1} f(\xi) d\xi$ may be represented by the weighted sum of the values of $f(\xi)$ at the Gauss points. The position and weightings of the Gauss points relate to the chosen number, as described in Table A.1.

n	$\pm\xi$	W
1	0	2.0
2	$1/\sqrt{3}$	1.0
3	0	8/9
	$\sqrt{3/5}$	5/9

Table A.1: Gauss point positions and weightings.

The technique is exact for $2n - 1$ polynomials where n is the chosen number of Gauss points.